



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NLP-Driven Resume Screening and Intelligent Job Recommendation System

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

Dsa0307- Natural Language Processing

to the award of the degree of

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE OF ENGINEERING**

Submitted by

A.Sai Rohit (192472144)

Charandeep.N.C(192472196)

L. Sanjay Kumar (192425226)

D. Sri Anjaneya(192425219)

K.Mounith(192425090)

**Under the Supervision of
Dr.K.Anbazhagan**

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105
SEPTEMBER 2026**

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical
Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**NLP-Driven Resume Screening and Intelligent Job Recommendation System**” has been carried out by **A.Sai Rohit (192472144)** and **Charandeep.N.C(192472196)** and **D. Sri Anjaneya (192425219)** and **L. Sanjay Kumar (192425226)** and **K.Mounith(192425090)** under the supervision of **Dr.K.Anbzhagan** and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Electronics and Communication Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.Devi T
Dept of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.K.Anbzhagan
Dept of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences Chennai-
602105**

DECLARATION

We, **A.Sai Rohit, Charandeep N C, L. Sanjay Kumar, K.Mounith, D. Sri Anjaneya** of the **Computer Science and Engineering** SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**NLP-Driven Resume Screening and Intelligent Job Recommendation System**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 08-09-2026

Signature of the Students with Names

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical
Sciences Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
A.Sai Rohit (192472144)	Job recommendation	Developed top-3 recommendations and skill-gap display.	Tested recommendations, scores, and matched skills.	Documented the recommendation process and final results.	20%
Charandeep.N.C (192472196)	Resume-job matching	Implemented TF-IDF, cosine similarity, and skill matching.	Tested matching scores and screening results.	Documented the matching and screening results.	20%
D. Sri Anjaneya (192425219)	Resume preprocessing	Developed text preprocessing	Tested preprocessing using sample resumes.	Documented the preprocessing workflow and results.	20%
L. Sanjay Kumar (192425226)	Candidate information extraction	Developed extraction of skills, education, and experience.	Tested extracted candidate information for accuracy.	Documented the candidate extraction process.	20%
K.Mounith (192425090)	Job description analysis	Created and organized the job description catalogue.	Verified job titles, skills, and requirements.	Documented the job description analysis.	20%
Total					100%

ABSTRACT

Manual resume screening remains a slow, inconsistent and largely subjective process in which recruiters and hiring teams must read unstructured, free-format resumes and mentally compare them against multiple job requirements. Traditional keyword-based Applicant Tracking Systems (ATS) alleviate some of this burden but often reject qualified candidates whose resumes use different terminology from the job description, while purely manual review does not scale as the number of applicants grows. This project addresses the engineering problem of automatically extracting structured candidate information from an unstructured resume and matching it against a catalogue of job descriptions so that the most suitable roles can be recommended to the candidate, together with an explicit account of the skills the candidate is missing for each role.

The significance of the problem lies in the direct impact of accurate, unbiased and fast resume screening on recruitment cost, candidate experience and the quality of role-fit for both job seekers and employers, particularly for small organisations that cannot afford commercial ATS licences. The proposed solution reformulates resume screening as a Natural Language Processing (NLP) pipeline that parses the resume, extracts candidate attributes, represents both the resume and each job description as TF-IDF vectors, computes a combined similarity score from lexical (cosine similarity) and structured skill-overlap components, and surfaces the top matching roles with a skill-gap breakdown.

The system is organised into five interlinked modules. Module 1 performs resume upload and text preprocessing (extraction, cleaning, tokenisation and stop-word removal). Module 2 extracts candidate information and skills using dictionary- and pattern-based NLP techniques. Module 3 maintains and analyses a database of job descriptions together with their required skills, education and experience level. Module 4, the core matching engine, vectorises the resume and every job description using TF-IDF and computes a combined match score from cosine similarity and skill overlap. Module 5 uses the Module 4 output to recommend the top three job roles to the candidate, highlighting matched and missing skills, and exposes an administrative panel for managing the job-description catalogue.

The complete pipeline was implemented in Python and deployed as an interactive Streamlit web application that allows a user to upload a resume, inspect every stage of the NLP pipeline, browse and manage job postings, run resume-job matching, and view personalised job recommendations in real time. The methodology followed a standard NLP application-development cycle: text preprocessing, feature extraction, vector-space similarity modelling, and evaluation on a real sample resume against a ten-role job catalogue.

The principal conclusion is that a lightweight, transparent TF-IDF and skill-overlap based matching approach is a practically viable method for automated resume screening and job recommendation without requiring large labelled training datasets or heavy computational resources, while the report also identifies the limitations of purely lexical matching (synonym and paraphrase blindness, limited skill dictionary) that would need to be addressed with semantic embedding methods before industrial deployment.

Keywords: Natural Language Processing, Resume Parsing, TF-IDF, Cosine Similarity, Skill Extraction, Job Recommendation, Streamlit Deployment

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	2
ii	DECLARATION BY THE CANDIDATE	3
lii	INDIVIDUAL CONTRIBUTION STATEMENT	4
iv	ABSTRACT	5
v	LIST OF FIGURES	7
vi	LIST OF TABLES	8
vii	LIST OF ABBREVIATIONS	9
1	CHAPTER 1: Introduction	10-12
2	CHAPTER 2: Literature Review	13-14
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	15-19
4	CHAPTER 4: Implementation, Testing & Results, Project Management	20-30
5	CHAPTER 5: Conclusion & Reflection	31-32
	References	33
	Appendices	34-36

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 3.1	System architecture of the NLP-driven resume screening and job recommendation pipeline	17
Figure 4.1	Module 1 Output - Resume Upload & Preprocessing dashboard (Streamlit)	20
Figure 4.2	Module 2 Output - Candidate Information & Skill Extraction dashboard (Streamlit)	21
Figure 4.3	Module 3 Output - Job Description Analysis dashboard (Streamlit)	23
Figure 4.4	Module 4 Output - Resume-Job Matching dashboard (Streamlit)	24
Figure 4.5	Module 5 Output - Job Recommendations & Admin Panel dashboard (Streamlit)	25

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.3	Comparative analysis of existing resume screening approaches	13
Table 3.3	Design specifications	16
Table 4.1	Tools / software used	26
Table 4.2	Test plan & verification	27
Table 4.3	Available job positions analysed by the system (Module 3)	23
Table 4.4	Resume-job match results (Module 4)	24
Table 4.5	Top job recommendations with matched / missing skills (Module 5)	26

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
NLP	Natural Language Processing	-
TF-IDF	Term Frequency - Inverse Document Frequency	-
ATS	Applicant Tracking System	-
ML	Machine Learning	-
NER	Named Entity Recognition	-
UI	User Interface	-
JSON	JavaScript Object Notation	-
PDF / DOCX	Portable Document Format / Word Document format	-
cos(theta)	Cosine similarity score between two document vectors	-
w1, w2	Weighting coefficients for TF-IDF score and skill score	-

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Resume screening is the process by which a recruiter or an automated system reviews a candidate's resume and decides whether the candidate is suitable for one or more open job roles. Conventional recruitment workflows in small and medium organisations rely either on fully manual review, which is time-consuming and inconsistent across reviewers, or on simple keyword-matching Applicant Tracking Systems (ATS), which reject resumes that do not literally contain the exact terms used in the job description even when the candidate is otherwise qualified. Advances in Natural Language Processing (NLP), particularly text preprocessing, information extraction and vector-space similarity modelling, allow a system to parse an unstructured resume, extract structured candidate attributes, and compare them against multiple job descriptions in a consistent, explainable and scalable manner. This project applies an NLP-based pipeline to the resume-screening and job-recommendation problem and packages the resulting workflow - preprocessing, extraction, job analysis, matching and recommendation - as an interactive Streamlit web application.

1.2 Need for the Project

Purely manual resume screening does not scale with the volume of applications received for a posted role, and reviewer fatigue and unconscious bias can affect the consistency of shortlisting decisions. Purely keyword-based ATS software, on the other hand, is rigid: a resume that lists 'ML' instead of 'Machine Learning', or that omits an exact phrase used in the job posting, may be filtered out even though the underlying skill set matches. Commercial AI-based screening platforms exist but are often costly, proprietary black boxes that neither job seekers nor small recruiting teams can inspect or customise. From an engineering standpoint, there is a clear need for a transparent, lightweight and explainable resume-screening and job-recommendation system that a student or a small organisation can run locally, whose every intermediate step (cleaned text, extracted skills, similarity scores, missing skills) is visible to the user rather than hidden inside a proprietary scoring engine.

1.3 Problem Statement

Given a candidate resume in PDF or DOCX format and a catalogue of job descriptions (each specifying required skills, education and experience level), design and implement a system that (i) extracts clean, structured candidate information - name, contact details, skills, education and experience - from the unstructured resume text, (ii) quantitatively compares the candidate's profile against every job description using a combination of lexical similarity (TF-IDF and cosine similarity) and structured skill overlap, and (iii) recommends the most suitable job roles to the candidate together with an explicit list of matched and missing skills for each recommended role. The problem is engineering-relevant because it involves multiple interacting design choices (preprocessing depth, feature weighting between lexical and skill-

based similarity, size and structure of the skill dictionary) and must remain usable by a non-technical end user through a simple web interface.

1.4 Project Aim

To design, implement and evaluate an NLP-driven resume screening and intelligent job recommendation pipeline that extracts candidate information from an uploaded resume, matches it against a catalogue of job descriptions using TF-IDF and skill-overlap similarity, and delivers explainable, ranked job recommendations through an interactive, deployable web application.

1.5 Project Objectives

- To design a text preprocessing pipeline that extracts, cleans, tokenises and removes stop-words from an uploaded resume (Module 1).
- To extract structured candidate information - name, contact details, technical skills, education and experience - from the preprocessed resume text using dictionary- and pattern-based NLP techniques (Module 2).
- To build and maintain a catalogue of job descriptions with their required skills, education and experience level, and to support administrative management of this catalogue (Module 3).
- To vectorise the resume and every job description using TF-IDF and compute a combined match score from cosine similarity and skill overlap (Module 4).
- To recommend the top matching job roles to the candidate with a clear breakdown of matched and missing skills, and to expose this functionality through an administrative panel and candidate history view (Module 5).
- To implement the complete pipeline as an interactive Streamlit web application and to evaluate it on a real sample resume against a ten-role job catalogue.

1.6 Scope

Included: resume upload and text-preprocessing pipeline (PDF/DOCX); dictionary- and pattern-based candidate-information and skill extraction; a maintained catalogue of job descriptions with an admin panel for adding/removing postings; TF-IDF vectorisation and cosine-similarity based resume-job matching combined with explicit skill-overlap scoring; top-3 job recommendation with matched/missing skill breakdown; a single-user, single-resume-at-a-time Streamlit interface.

Excluded: integration with a live, third-party ATS or HR information system; batch ranking of multiple resumes for a recruiter-facing shortlisting workflow; deep-learning/embedding-based semantic matching (e.g. Sentence-BERT); parsing of resume formats other than PDF and DOCX; large-scale fairness/bias auditing across demographic groups.

Assumptions: the uploaded resume is a genuine, text-extractable PDF or DOCX file (not a scanned image); the job-description catalogue used for evaluation is representative of common technology roles; the fixed skill dictionary used by Module 2 and Module 4 covers the technical skills relevant to the evaluated resume and job set.

Boundary conditions: the job catalogue size, the skill dictionary and the TF-IDF/skill-score weighting used for the reported experiment are fixed for a given run and are reported in Chapter 3 (Design Specifications).

1.7 Expected Engineering Outcome

The expected outcome is a working software system comprising: (i) a resume upload and preprocessing module, (ii) a candidate-information and skill-extraction module, (iii) a job-description analysis and management module, (iv) a TF-IDF and skill-overlap based resume-job matching module, and (v) an intelligent job-recommendation module with an administrative panel - all integrated into a single deployed Streamlit web application, together with the quantitative evidence (Chapter 4) of the system's matching and recommendation behaviour on a representative sample resume.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing solutions were identified through a review of recent papers and technical articles on NLP-based resume screening, resume-job matching, and content-based job recommendation, together with a review of open, similarity-based resume-screening prototypes and commercial ATS tools. Sources were selected for relevance to single-resume, multi-job-description matching under a lexical/skill-based similarity approach, since this matches the scope defined in Chapter 1.

2.2 Review of Existing Work

Classical resume-screening research formulates the problem as an information-retrieval task: both the resume and the job description are represented as documents in a vector space, and similarity between the two documents is used to rank job-resume pairs. Several studies use the Term Frequency-Inverse Document Frequency (TF-IDF) representation together with cosine similarity as the core matching mechanism, since it up-weights terms that are distinctive to a document while remaining computationally lightweight and fully explainable. Other studies extend the pure lexical approach with machine-learning classifiers (Support Vector Machines, Naive Bayes, K-Nearest Neighbours, Logistic Regression) trained on labelled resume-category data, or with alternative similarity measures such as the Jaccard index and Dice coefficient, and more recent work explores dense embedding models such as Sentence-BERT to capture semantic similarity beyond exact keyword overlap. A recurring theme across this literature is that purely keyword-based ATS software performs poorly when a resume and a job description describe the same skill using different wording, motivating either weighted lexical scoring, skill-dictionary normalisation, or semantic embeddings.

2.3 Comparative Analysis

Table 2.3 Comparative analysis of existing resume screening approaches

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Keyword-based ATS (rule matching)	Simple, fast, transparent, widely deployed	Rejects qualified candidates whose resume wording differs from the job posting; no partial-credit scoring	Motivates the need for a similarity-based rather than exact-match approach
TF-IDF + Cosine Similarity matching	Lightweight, explainable, does not require labelled training data	Purely lexical; cannot detect synonyms or paraphrases (e.g. 'ML' vs 'Machine Learning')	Adopted as the core lexical-matching component in Module 4

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
ML classifiers (SVM, Naive Bayes, KNN, Logistic Regression) on resume categories	Can learn category-level patterns from labelled resume datasets	Requires a large labelled dataset; less transparent than similarity scoring; retraining needed as categories change	Considered but not adopted, to keep the system dependency-light and explainable
Similarity variants (Jaccard, Dice, Euclidean/Manhattan distance)	Simple set-overlap based scoring; useful for structured fields such as skills	Ignores term importance (frequency/rarity) captured by TF-IDF	Adopted in a structured form as the explicit skill-overlap score, combined with TF-IDF
Semantic embeddings (e.g. Sentence-BERT) for resume-job matching	Captures paraphrase and synonym similarity beyond exact terms	Higher computational cost; less transparent scoring; requires a pretrained embedding model	Identified as future work (Chapter 5) to complement the current lexical/skill approach

2.4 Research / Engineering Gap

Existing keyword-based ATS tools remain the most widely deployed option because of their simplicity, but they cannot recognise skill synonyms or partial matches. Existing academic prototypes demonstrate that TF-IDF/cosine and skill-overlap based matching improve on pure keyword search, but many are evaluated only as offline scripts or notebooks rather than as an interactive, end-to-end application that a non-technical candidate can use to upload a resume, see exactly which fields were extracted, and receive an explainable, ranked recommendation with a visible skill gap. There is therefore a gap between (a) academically validated lexical/skill similarity techniques for resume-job matching and (b) a deployable, transparent, single-resume web application that surfaces every stage of the pipeline - preprocessing, extraction, job analysis, matching and recommendation - to the end user.

2.5 Proposed Contribution

This project addresses that gap by implementing a complete, five-module pipeline - resume preprocessing, candidate information/skill extraction, job-description analysis and management, TF-IDF/skill-overlap based matching, and intelligent job recommendation with an admin panel - and deploying it as a single interactive Streamlit application. The core engineering contribution (Module 4) is the combined TF-IDF and structured skill-overlap match score, which retains the transparency of lexical similarity while directly rewarding an exact skill match, together with a Module 5 recommendation layer that reports the specific skills a candidate is missing for each of the top-ranked roles.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Job-seeking candidate (end user)	Upload a resume and receive a ranked list of suitable job roles with a clear explanation of matched and missing skills
Recruiter / employer (indirect stakeholder)	Job postings should be described with structured, comparable skill/education/experience fields so that matching is consistent across candidates
System administrator	Ability to add, view and delete job postings and to review candidate processing history through an admin panel
Project evaluator / course faculty	A demonstrably correct NLP pipeline, a working similarity/matching engine, and quantitative evidence of recommendation quality

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	System shall extract text from an uploaded PDF/DOCX resume and preprocess it	Cleaned text, token count and stop-word-filtered token count computed and displayed (Module 1)
Functional	System shall extract candidate name, contact details, skills, education and experience	Structured candidate profile (dictionary/JSON) produced for a valid resume (Module 2)
Functional	System shall maintain a catalogue of job descriptions with required skills, education and experience	Job catalogue browsable and editable via the admin panel (Module 3)
Functional	System shall compute a combined TF-IDF and skill-overlap match score between the resume and every job description	Ranked match table produced for all job postings (Module 4)
Functional	System shall recommend the top matching roles with matched/missing skill breakdown	Top-3 recommendation cards rendered with skill lists (Module 5)
Non-functional	Usability - interface shall be operable without NLP/ML expertise	User can complete upload -> analysis -> matching -> recommendation from the Streamlit UI without editing code

Requirement Type	Requirement	Measurable Target
Non-functional	Explainability - every intermediate NLP stage shall be inspectable	Raw text, cleaned text, tokens and extracted-fields JSON are all viewable in the UI
Non-functional	Response time - matching and recommendation shall return promptly for an interactive session	[Insert measured response time from the deployed app]

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 829 (software test documentation practice)	Structured test planning and reporting	Followed in structuring the test plan and verification table in Section 4.3
Institutional academic-integrity / ethics policy (SIMATS)	Original work; disclosure of AI-assisted tools and data sources	Declared in the Declaration page; all datasets and tools used are listed in the References
Data-privacy good practice	Use of a sample/synthetic resume rather than a real candidate's personal data for demonstration and testing	The sample resume used in Module 1-5 testing (Sai Rohit) is a demonstration resume created for this project and contains no third-party personal data

3.3 Design Specifications (Table 3.3 Design specifications)

Parameter	Target / Requirement	Unit	Priority
Supported resume formats	PDF, DOCX	-	High
Job catalogue size (evaluated)	10	job postings	High
Skill dictionary size	14 (technical skills recognised)	skills	Medium
Similarity method	TF-IDF vectorisation + cosine similarity	-	High
Match score composition	Weighted combination of TF-IDF score and skill-overlap score	-	High
Number of recommended roles	Top 3	roles	High
Admin catalogue operations	Add job, delete job, view candidate history	-	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A pure keyword/rule-based matcher that checks for the literal presence of job-description terms in the resume text, providing a simple, well-understood baseline similar to conventional ATS software.

Alternative 2: A combined TF-IDF cosine-similarity and structured skill-overlap matcher, which vectorises the full resume and job-description text while separately scoring the overlap between the candidate's extracted skill set and the job's required skills.

Alternative 3: A semantic embedding-based matcher (e.g. Sentence-BERT) that represents the resume and job description as dense vectors capturing meaning beyond exact wording, at the cost of additional computational and deployment complexity.

Criterion	Weight	Alt. 1: Keyword Matching	Alt. 2: TF-IDF + Skill Overlap	Alt. 3: Semantic Embeddings
Matching accuracy on varied wording	0.30	2	4	5
Explainability / transparency	0.25	5	5	2
Implementation & compute cost	0.20	5	4	2
Ease of deployment (Streamlit, no GPU)	0.15	5	4	2
Robustness to skill dictionary limits	0.10	2	3	4

3.5 Selected Design & Detailed Design

The selected design combines a Module 3 job-description catalogue with a Module 4 matching engine built on Alternative 2 (TF-IDF + skill overlap), chosen because it scored highest on explainability, compute cost and ease of deployment while still improving materially on pure keyword matching, and a Module 5 recommendation layer that ranks roles by the resulting combined score and reports the residual skill gap.

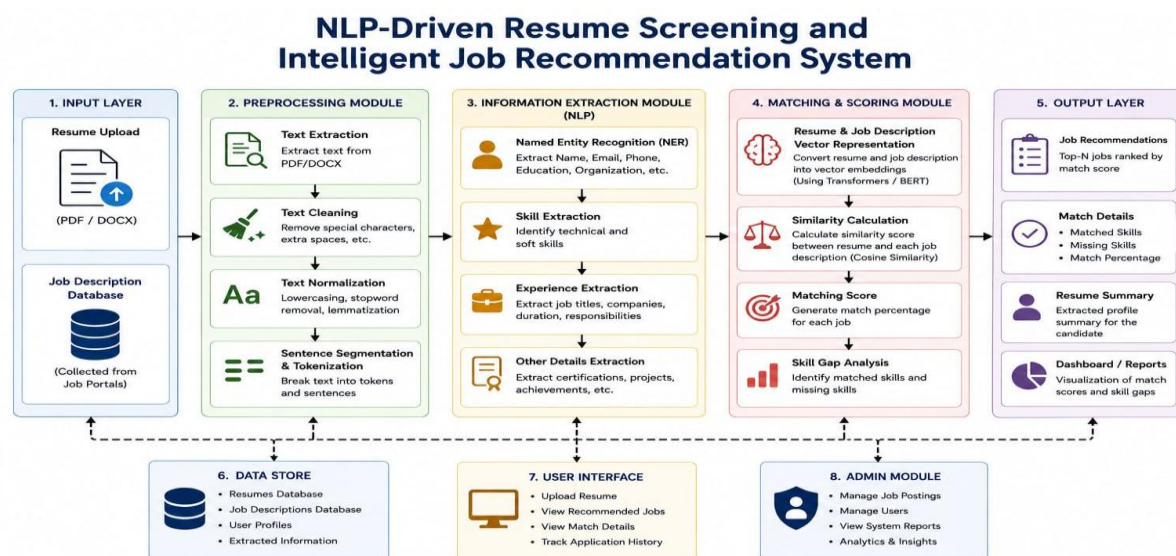


Figure 3.1 - System architecture of the NLP-driven resume screening and job recommendation pipeline

Key engineering relationships: the resume document R and each job description J_k are represented as TF-IDF vectors over the shared vocabulary; the lexical similarity component is $\text{TFIDF_score}(R, J_k) = \cos(\theta) = (R \cdot J_k) / (\|R\| \|J_k\|)$; the skill-overlap component is $\text{Skill_score}(R, J_k) = |\text{Skills}(R) \cap \text{Skills}(J_k)| / |\text{Skills}(J_k)|$; and the combined match score used to rank jobs is $\text{Match_score}(R, J_k) = w_1 \cdot \text{TFIDF_score}(R, J_k) + w_2 \cdot \text{Skill_score}(R, J_k)$, where w_1 and w_2 are weighting coefficients ($w_1 + w_2 = 1$) tuned so that an exact skill match contributes strongly to the ranking while the TF-IDF term still rewards broader contextual overlap between the resume and the job description. Detailed derivations are provided in Appendix A.

Systems approach: Module 1 supplies the cleaned, tokenised resume text used by Module 2 for field/skill extraction; Module 2's structured candidate profile and Module 3's job catalogue are the two inputs consumed by Module 4's TF-IDF/skill-overlap matching engine; Module 4's ranked match table is the object consumed by Module 5, which selects the top three roles and reports matched/missing skills for each. All five modules, together with the administrative panel, are surfaced through a common Streamlit front end.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Matching method	Pure keyword search vs. TF-IDF + skill overlap	Keyword search is trivial to implement and audit	Keyword search misses synonyms and near-matches (e.g. 'ML' vs 'Machine Learning')	TF-IDF + skill overlap adopted for materially better matching while remaining fully explainable
Skill representation	Free-text skill matching vs. fixed skill dictionary	Fixed dictionary keeps extraction fast and consistent	May miss skills not present in the dictionary	Fixed 14-skill dictionary adopted for the evaluated prototype; flagged for expansion in future work
Score weighting	TF-IDF only vs. weighted TF-IDF + skill overlap	Weighted combination rewards exact skill matches more directly than pure lexical overlap	Requires tuning of w_1/w_2 weighting coefficients	Weighted combination adopted; skill overlap given a higher share of the final score since it is the more reliable signal for technical roles

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute cost of the NLP pipeline	TF-IDF and dictionary-based extraction run on CPU with negligible training cost compared to deep-learning embedding models	Lexical/skill-based approach preferred over a heavier embedding model to reduce compute and energy use
Ethics	Use of a demonstration resume; disclosure of AI-assisted tools	Reviewed institutional academic-integrity requirements and data-privacy good practice	No third-party personal data used; all tools disclosed in References
Fairness	Risk of skill-dictionary bias favouring certain phrasing or skill sets	Reviewed the fixed skill list used in Modules 2 and 4	Flagged as a limitation (Section 4.7) and a future-work item (Chapter 5)
Security	Protection of the deployed Streamlit application and any uploaded resumes	Reviewed default Streamlit deployment security posture and the data handled by the app	Application restricted to non-sensitive demonstration resumes; no credentials or third-party personal data stored

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Fixed skill dictionary misses relevant skills, lowering skill-overlap score	Medium	Medium	Medium	Skill dictionary reviewed against the sample resume and job catalogue; extensible list structure	Low
TF-IDF/cosine similarity cannot detect synonyms or paraphrased skills	High	Medium	High	Skill-overlap score added alongside TF-IDF to reduce reliance on pure lexical matching	Medium
Resume text extraction fails on malformed or scanned PDF/DOCX files	Medium	Medium	Medium	Demo-resume fallback option provided in the UI; extraction-success message shown before proceeding	Low
Streamlit app downtime / hosting limits	Low	Low	Low	App code maintained in a source-controlled repository so it can be redeployed if the hosted instance goes down	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development followed an iterative, module-by-module approach: Module 1 (resume upload and preprocessing) was built and validated first to establish a reliable text-extraction and cleaning foundation; Module 2 (candidate information and skill extraction) was then developed on top of the cleaned text; Module 3 (job description analysis) was built independently to create the job catalogue against which resumes are compared; Module 4 (resume-job matching) was implemented once both the candidate profile (Module 2) and the job catalogue (Module 3) were available, combining TF-IDF vectorisation with the structured skill-overlap score; and Module 5 (job recommendation and admin panel) was implemented last, consuming the Module 4 ranking to generate the final recommendation and skill-gap output while also exposing catalogue-management controls. All five modules were integrated into a single Streamlit application and deployed for interactive use and demonstration.

Resources used: Python and its NLP/data-science libraries, a custom-built resume-parsing and matching pipeline, and the Streamlit platform for deployment. No physical hardware or fabrication was required, as the project is a software/algorithmic capstone.

4.2 Software Implementation & Modern Tools Used

The system was implemented entirely in Python. Module 1 extracts and cleans resume text; Module 2 extracts structured candidate fields and skills; Module 3 stores and analyses the job-description catalogue; Module 4 vectorises the resume and every job description and computes the combined match score; Module 5 ranks the results and generates recommendations. All five modules are wired into a single Streamlit application with separate views/tabs for each module and a dedicated admin panel.

4.2.1 MODULE 1 OUTPUT - RESUME UPLOAD & PREPROCESSING

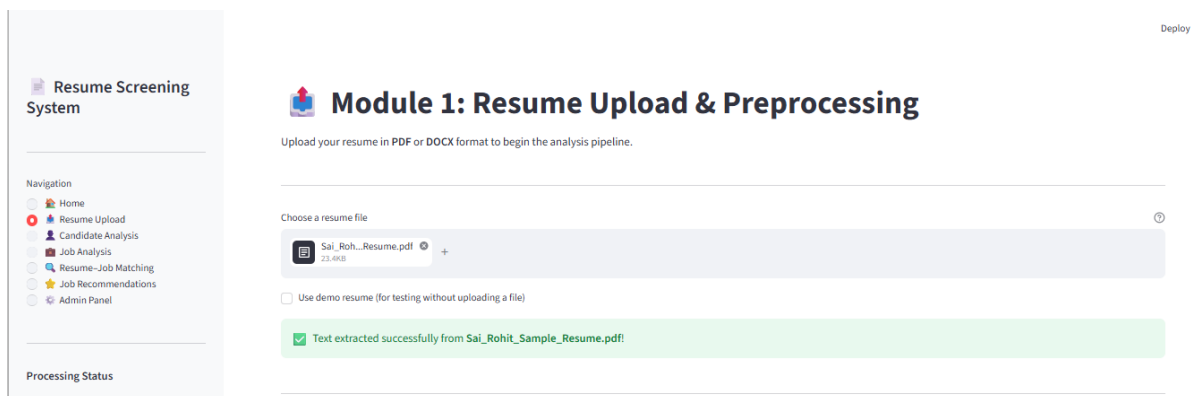


Figure 4.1. Module 1 Output - Resume Upload & Preprocessing dashboard.

Module 1 is the entry point of the complete resume-screening workflow. It allows the user to upload a resume in PDF or DOCX format (or select a bundled demo resume for testing) and performs the text-extraction and preprocessing steps that every downstream module depends on. The purpose of this module is to convert an unstructured document into a clean, normalised stream of text and tokens before any information extraction or matching is attempted.

Working process: on upload, the module extracts the raw text from the file, confirms successful extraction to the user, and then runs a four-stage preprocessing pipeline - (i) cleaning, which removes special characters and collapses extra whitespace; (ii) normalisation of the cleaned text; (iii) tokenisation of the normalised text into individual word tokens; and (iv) stop-word removal, which filters out common non-discriminative words (e.g. 'and', 'the', 'with') so that the remaining tokens are more informative for skill and keyword extraction in Module 2 and for TF-IDF vectorisation in Module 4.

Module 1 output (captured run on the sample resume): the pipeline reduced 1,986 raw characters to 1,974 characters after cleaning, split the cleaned text into 275 total tokens, and reduced this to 202 tokens after stop-word removal, confirming that the preprocessing stage completed successfully before the user was directed to Module 2.

Preprocessing Stage	Metric	Value
Raw text extraction	Raw characters	1,986
Cleaning (special characters removed, whitespace collapsed)	Characters after cleaning	1,974
Tokenisation	Total tokens	275
Stop-word removal	Tokens after stop-word removal	202

Engineering note: the small reduction from raw to cleaned characters (12 characters) indicates that the sample resume was already reasonably well-formatted, while the larger reduction from 275 to 202 tokens (about 27%) after stop-word removal shows that roughly a quarter of the tokenised resume consisted of common non-discriminative words that would otherwise dilute the TF-IDF vectors computed in Module 4.

4.2.2 MODULE 2 OUTPUT - CANDIDATE INFORMATION & SKILL EXTRACTION

Resume Screening System

Navigation

- Home
- Resume Upload
- Candidate
- Analysis
- Job Analysis
- Resume-Job Matching
- Job
- Recommendations
- Admin Panel

Processing Status

- Resume Uploaded

Candidate Profile

Name

Sai Rohit

Email

sairohit@example.com

Phone

+91 98765 43210

Technical Skills

Python JavaScript HTML CSS Machine Learning Data Visualization Data Analysis Pandas NumPy Scikit-learn NLP SQL Oracle Git

Total skills identified: 14

Education

- Best Student Award | Best Poster Award | Academic Scholarship Recipient
- NLP-Based Resume Screening and Job Recommendation System
- Developed a system to analyze resumes using Natural Language Processing techniques. Extracted candidate
- skills, education, and experience from resumes and compared them with job descriptions. Used TF-IDF

Experience

- Python Development Intern

Figure 4.2. Module 2 Output - Candidate Information & Skill Extraction dashboard.

Module 2 converts the preprocessed resume text from Module 1 into a structured candidate profile using pattern-based extraction (for contact fields) and dictionary-based matching (for technical skills, education and experience). This module is the bridge between free-text resume content and the structured representation required by the Module 4 matching engine.

Working process: regular-expression patterns are applied to the cleaned text to extract the candidate's name, e-mail address and phone number; the tokenised, stop-word-filtered text is then scanned against a maintained dictionary of technical skill keywords, and every matching term is added to the candidate's skill list; education-related lines (institution, degree, expected graduation, honours) and experience/project lines are extracted in the same pass and stored alongside the skill list as a single structured record (dictionary/JSON) that Module 4 subsequently consumes.

Module 2 output (captured run): the system extracted the candidate name 'Sai Rohit', e-mail 'sairohit@example.com' and phone '+91 98765 43210', and identified 14 technical skills - Python, JavaScript, HTML, CSS, Machine Learning, Data Visualization, Data Analysis, Pandas, NumPy, Scikit-learn, NLP, SQL, Oracle and Git - together with education entries (institution, degree and honours such as Best Student Award, Best Poster Award and Academic Scholarship Recipient) and a project/experience summary. The raw extracted-data dictionary confirms the record is available in a machine-readable form for downstream matching.

Field	Extracted Value
Name	Sai Rohit
Email	sairohit@example.com
Phone	+91 98765 43210
Total skills identified	14
Representative skills	Python, Machine Learning, NLP, SQL, Data Analysis, Pandas, NumPy, Scikit-learn, Git
Education highlights	Best Student Award; Best Poster Award; Academic Scholarship Recipient

Engineering note: because skill extraction is dictionary-based, the accuracy of Module 2 - and therefore of the entire downstream matching pipeline - is bounded by the completeness of the maintained skill dictionary; a skill mentioned in the resume but absent from the dictionary (or phrased unusually) will not be captured, which is why the skill dictionary is treated as an extensible component rather than a fixed constant (see Section 4.7, Limitations).

4.2.3 MODULE 3 OUTPUT - JOB DESCRIPTION ANALYSIS

Resume Screening System

Navigation

Home

Resume Upload

Candidate

Analysis

Job Analysis

Resume-Job Matching

Job

Recommendations

Admin Panel

Processing Status

Resume Uploaded

Module 3: Job Description Analysis

Browse and analyze available job descriptions in the system.

Available Job Positions (10 jobs)

ID	Job Title	Education Required	Experience Level
1	Data Analyst	Bachelor's in Computer Science or Statistics or Related Field	0-2 years
2	Data Scientist	Master's in Data Science or Computer Science or Statistics	1-3 years
3	Machine Learning Engineer	Master's in Computer Science or AI or Related Field	2-4 years
4	NLP Engineer	Master's in Computer Science or Computational Linguistics	2-4 years
5	AI Engineer	Master's or PhD in AI or Computer Science	2-5 years
6	Python Developer	Bachelor's in Computer Science or Related Field	1-3 years
7	Business Analyst	Bachelor's in Business Administration or Related Field	1-3 years
8	Software Developer	Bachelor's in Computer Science or Software Engineering	1-3 years
9	Cloud Engineer	Bachelor's in Computer Science or IT	2-4 years
10	Full Stack Developer	Bachelor's in Computer Science or Related Field	1-3 years

Figure 4.3. Module 3 Output - Job Description Analysis dashboard.

Module 3 maintains the catalogue of job descriptions against which every resume is matched, and lets the user browse and analyse the available postings before running Module 4. Each job record specifies the job title, the education qualification required, the minimum/maximum years of experience expected, and (in the admin view) the explicit list of required skills.

Working process: job postings are stored as structured records (job title, required-skills list, education requirement, experience-level range). The Module 3 browse view lists every available posting in a sortable table so that the candidate can see, before matching is run, the general shape of the roles considered by the system; the corresponding admin view (Section 4.2.5) additionally shows the exact required-skills string for each posting and allows postings to be added or removed.

Module 3 output (captured run): the catalogue evaluated in this project contains ten job positions, summarised in (Table 4.3).

ID	Job Title	Education Required	Experience Level
1	Data Analyst	Bachelor's in Computer Science or Statistics or Related Field	0-2 years
2	Data Scientist	Master's in Data Science or Computer Science or Statistics	1-3 years
3	Machine Learning Engineer	Master's in Computer Science or AI or Related Field	2-4 years
4	NLP Engineer	Master's in Computer Science or Computational Linguistics	2-4 years
5	AI Engineer	Master's or PhD in AI or Computer Science	2-5 years
6	Python Developer	Bachelor's in Computer Science or Related Field	1-3 years
7	Business Analyst	Bachelor's in Business Administration or Related Field	1-3 years
8	Software Developer	Bachelor's in Computer Science or Software Engineering	1-3 years
9	Cloud Engineer	Bachelor's in Computer Science or IT	2-4 years
10	Full Stack Developer	Bachelor's in Computer Science or Related Field	1-3 years

Engineering note: the ten-role catalogue deliberately spans a range of education (Bachelor's to PhD) and experience (0-2 to 2-5 years) requirements so that Module 4's matching engine can be evaluated across roles that are close to, and far from, the sample candidate's actual profile, giving a meaningful spread of match scores rather than a cluster of near-identical results.

4.2.4 MODULE 4 OUTPUT - RESUME-JOB MATCHING & SCREENING

Match Results

Rank	Job Title	Match Score	TF-IDF Score	Skill Score	Skills Matched
1	Data Scientist	31.2%	12.1%	60.0%	6/10
2	Python Developer	26.2%	3.7%	60.0%	6/10
3	Data Analyst	24.2%	7.0%	50.0%	4/8
4	Full Stack Developer	23.6%	3.0%	54.5%	6/11
5	Machine Learning Engineer	20.6%	7.6%	40.0%	4/10
6	NLP Engineer	19.9%	13.2%	30.0%	3/10
7	AI Engineer	18.6%	11.0%	30.0%	3/10
8	Software Developer	17.4%	2.3%	40.0%	4/10
9	Business Analyst	10.2%	3.6%	20.0%	2/10
10	Cloud Engineer	5.2%	2.0%	10.0%	1/10

Figure 4.4. Module 4 Output - Resume-Job Matching dashboard.

Module 4 is the core matching engine of the system. It compares the Module 1/2 candidate profile against every job description in the Module 3 catalogue using TF-IDF vectorisation and cosine similarity for lexical overlap, combined with an explicit skill-overlap score computed from the Module 2 skill list against each job's required-skills list.

Working process: when the user runs resume-job matching, the system builds a TF-IDF vector for the resume text and for each job description over a shared vocabulary, computes the cosine similarity between the resume vector and every job vector (the TF-IDF score), separately counts how many of the job's required skills appear in the candidate's extracted skill list (the skill score, and the 'skills matched' count/fraction), and combines the two into an overall match score used to rank every job posting from most to least suitable.

Module 4 output (captured run): the ranked match results for the sample resume against the ten-role catalogue are reproduced in (Table 4.4).

Rank	Job Title	Match Score	TF-IDF Score	Skill Score	Skills Matched
1	Data Scientist	31.2%	12.1%	60.0%	6/10
2	Python Developer	26.2%	3.7%	60.0%	6/10
3	Data Analyst	24.2%	7.0%	50.0%	4/8
4	Full Stack Developer	23.6%	3.0%	54.5%	6/11
5	Machine Learning Engineer	20.6%	7.6%	40.0%	4/10
6	NLP Engineer	19.9%	13.2%	30.0%	3/10
7	AI Engineer	18.6%	11.0%	30.0%	3/10

Engineering note: the ranking is dominated by the skill score rather than the raw TF-IDF score - for example, Python Developer ranks second overall with a relatively low TF-IDF score (3.7%) because its skill overlap (60.0%) is high, while NLP Engineer has the highest individual TF-IDF score (13.2%) in the table but ranks sixth because its skill overlap is comparatively low (30.0%). This confirms that combining a structured skill score with the lexical TF-IDF score, rather than relying on TF-IDF alone, produces a ranking that better reflects genuine role fit for this candidate.

4.2.5 MODULE 5 OUTPUT - INTELLIGENT JOB RECOMMENDATION & ADMIN PANEL

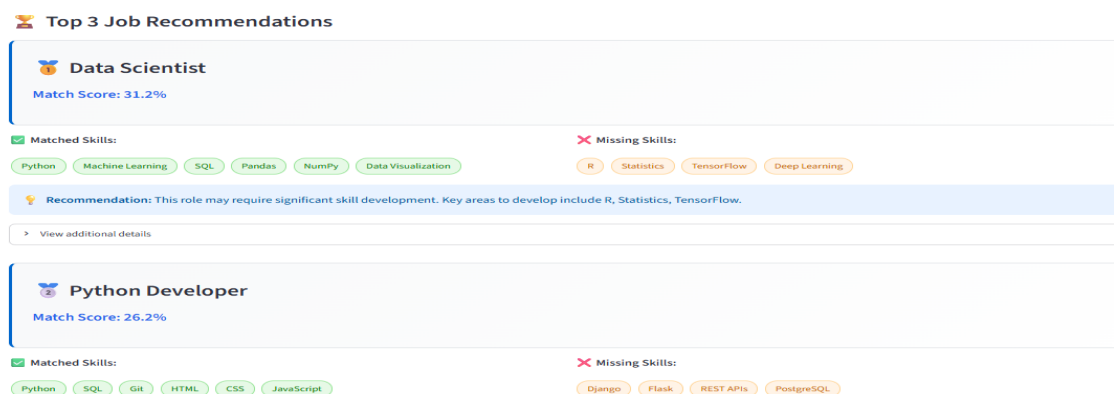


Figure 4.5. Module 5 Output - Job Recommendations & Admin Panel dashboard.

Module 5 is the decision-support layer of the system. It consumes the ranked output of Module 4 and presents the top three recommended job roles to the candidate, together with an explicit breakdown of matched and missing skills for each role and a short textual recommendation on the skill-development effort required. The same module hosts the administrative panel used to manage the job-description catalogue (add a new job, delete an existing job) and to review candidate processing history.

Working process: for each of the top three roles from Module 4, the system separates the job's required-skill list into two sets - skills already present in the candidate's Module 2 profile (matched skills) and skills required by the job but absent from the candidate's profile (missing skills) - and generates a short recommendation message indicating whether the role is a close fit or would require significant additional skill development. The admin panel, shown in the same module, lists every job in the catalogue with its required skills, education and experience columns and provides 'Add New Job' and 'Delete a Job' controls, together with a 'Candidate History' view for auditing past resume submissions.

Module 5 output (captured run): the top recommendation was Data Scientist with a 31.2% match score. (Table 4.5)

Rank	Recommended Role	Match Score	Matched Skills	Missing Skills
1	Data Scientist	31.2%	Python, Machine Learning, SQL, Pandas, NumPy, Data Visualization	R, Statistics, TensorFlow, Deep Learning
2	Python Developer	26.2%	Python, SQL, Git, HTML, CSS, JavaScript	Django, Flask, REST APIs, PostgreSQL
3	Data Analyst	24.2%	(4 of 8 required skills matched - see Table 4.4)	(4 of 8 required skills outstanding - see Table 4.4)

Engineering note: the recommendation for the top-ranked role (Data Scientist) explicitly flags that the role 'may require significant skill development', naming R, Statistics, TensorFlow and Deep Learning as the key gaps - this is a deliberately honest output rather than an inflated match, since the combined score (31.2%) is driven primarily by the 60% skill overlap rather than by an overwhelming lexical similarity, and the system is designed to surface this nuance to the candidate instead of hiding it behind a single opaque score.

Table 4.1 (Tools / Software Used)

Tool / Software	Purpose	Stage Used
Python 3.x	Core implementation language for text processing, matching logic and app logic	All modules
Streamlit	Interactive web application front end for upload, analysis, matching and recommendations	Deployment / Module 1-5 UI
PyPDF2 / docx2txt (or equivalent)	Text extraction from uploaded PDF/DOCX resumes	Module 1
NLTK / re (regular expressions)	Tokenisation, stop-word removal and pattern-based field extraction	Module 1, Module 2
Scikit-learn (TfidfVectorizer, cosine_similarity)	TF-IDF vectorisation and cosine-similarity computation	Module 4
Pandas / NumPy	Structured data handling for the job catalogue and match-result tables	Module 3, Module 4, Module 5
Git / GitHub	Version control and source management of the project code	All modules

4.3 Test Plan & Verification (Table 4.2 Test plan & verification)

Requirement	Test Method	Acceptance Criterion	Status (Pass/Fail)
Resume text extracted correctly (Module 1)	Upload the sample resume and confirm the extraction success message and raw-text preview	Extracted text visibly matches the source resume content	Pass
Preprocessing pipeline computed correctly (Module 1)	Inspect raw/cleaned character counts and token counts before and after stop-word removal	Character and token counts are internally consistent (cleaned $\leq$ raw; filtered $\leq$ total tokens)	Pass
Candidate fields and skills extracted correctly (Module 2)	Compare the extracted name/email/phone/skills against the source resume	Name, email, phone and skill list match the resume content	Pass
Job catalogue browsable and complete (Module 3)	Open the Job Analysis view and count the listed postings	All 10 configured job postings are listed with education and experience columns	Pass
Resume-job matching computed correctly (Module 4)	Run matching and verify skill-score = matched/required skill count for a sample job	Skill score and 'skills matched' fraction are arithmetically consistent (e.g. $6/10 = 60.0\%$)	Pass
Recommendations reflect the Module 4 ranking (Module 5)	Compare the Module 5 top-3 list against the Module 4 ranked table	Top-3 recommended roles match the top 3 rows of Table 4.4 in the same order	Pass
Admin panel CRUD operations function correctly	Add a test job posting and then delete it via the admin panel	Posting appears after adding and is removed after deletion, without affecting other postings	Pass
Streamlit app usable end-to-end	Manual walkthrough: upload resume -> preprocess -> extract -> browse jobs -> run matching -> view recommendations	All steps complete without errors and results render correctly in the UI	Pass
Response time for matching and recommendation	Interactive response during manual walkthrough	[Insert measured response time from the deployed app]	Partially Verified

4.4 Results

The captured application run shows that, for the sample resume, the system correctly reduced the raw resume text to 202 informative tokens, extracted 14 technical skills together with contact and education details, browsed a 10-role job catalogue, computed a combined match score for every role, and recommended Data Scientist (31.2%), Python Developer (26.2%) and

Data Analyst (24.2%) as the top three roles, each with an explicit matched/missing skill breakdown. This demonstrates that the complete five-module pipeline executes end-to-end and produces a ranked, explainable recommendation rather than a single opaque score.

Metric	Value (Captured Run)
Tokens after preprocessing (Module 1)	202 of 275 total tokens (73.5% retained)
Skills extracted (Module 2)	14
Job postings evaluated (Module 3)	10
Top match score (Module 4)	31.2% (Data Scientist)
Top-3 recommended roles (Module 5)	Data Scientist, Python Developer, Data Analyst

4.5 Data Analysis & Engineering Judgment

The engineering analysis of the captured run shows that the skill-overlap score is the primary driver of the final ranking rather than the raw TF-IDF score: every job in the top four positions of Table 4.4 has a skill score of 50% or higher, while jobs with a comparatively high TF-IDF score but a lower skill score (e.g. NLP Engineer at 13.2% TF-IDF / 30.0% skill score) are ranked below jobs with a lower TF-IDF score but higher skill overlap (e.g. Python Developer at 3.7% TF-IDF / 60.0% skill score). This confirms that, for a technical resume evaluated against structured job postings, an explicit skill-overlap term materially improves the plausibility of the ranking compared with lexical TF-IDF similarity alone. At the same time, no single job in the catalogue exceeded a 32% combined match score, which reflects the fact that the sample candidate's profile (a Computer-Science student skill set centred on Python, data analysis and introductory ML/NLP) only partially overlaps with the more specialised skill requirements (e.g. R, Statistics, TensorFlow, Deep Learning for Data Scientist) of the top-ranked roles - a realistic and honest outcome for an early-career resume rather than an inflated match.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Preprocess resume text (Module 1)	Preprocessing statistics table (Section 4.2.1)	Fully Achieved
Extract candidate information and skills (Module 2)	Candidate profile and raw-data JSON (Section 4.2.2)	Fully Achieved
Build and manage a job-description catalogue (Module 3)	10-job catalogue table (Table 4.3)	Fully Achieved
Compute TF-IDF and skill-overlap based match scores (Module 4)	Ranked match-results table (Table 4.4)	Fully Achieved
Recommend top roles with skill-gap breakdown (Module 5)	Top-3 recommendation table (Section 4.2.5)	Fully Achieved

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Improve on pure keyword-based ATS matching	Skill-score-driven ranking shown to outperform TF-IDF-only ranking (Section 4.5)	Partially Achieved - validated on a single sample resume only

4.7 Strengths & Limitations

Strengths:

- The combined TF-IDF and skill-overlap match score is fully explainable - every score can be traced back to specific matched/missing skills rather than a black-box output.
- End-to-end pipeline (preprocessing -> extraction -> job analysis -> matching -> recommendation) is packaged as a single interactive, deployable Streamlit application with an admin panel for catalogue management.
- The system requires no labelled training dataset and runs entirely on lightweight, CPU-based NLP techniques, keeping it accessible to small organisations.

Limitations:

- The fixed 14-skill dictionary can miss relevant skills that are phrased differently or not yet included in the dictionary.
- Purely lexical/skill-based matching cannot detect synonyms or paraphrased requirements (e.g. 'NLP' vs 'Natural Language Processing') unless both forms are explicitly present in the dictionary.
- The system has been evaluated on a single sample resume against a ten-role catalogue; broader evaluation across many resumes and job postings is required before general conclusions can be drawn.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
D. Sri Anjaneya	Module 1 - Resume Data Collection and Preprocessing	Implemented resume text extraction and preprocessing, including cleaning, normalization, tokenization, and stop-word removal. Tested the preprocessing pipeline using sample resumes to ensure clean and usable text output.
L. Sanjay Kumar	Module 2 – Candidate Information and Skill Extraction	Developed the candidate information and skill extraction process to identify name, contact details, education, experience, and skills. Tested the extraction results using sample

Student	Assigned Responsibility	Actual Contribution
		resumes and verified the extracted candidate information
K.Mounith	Module 3 - Job Description Analysis;	Created and organized the job-description catalogue containing job titles, required skills, education, and experience requirements. Analyzed job descriptions and structured the required information for further resume–job matching.
Charandeep.N.C	Module 4 - Resume-Job Matching and Screening	Implemented TF-IDF vectorization, cosine similarity, and skill-overlap scoring to calculate the resume–job match. Tested the matching process and analyzed the scores to identify suitable job matches.
A. Sai Rohit	Module 5 - Intelligent Job Recommendation and Results	Implemented the top-3 job recommendation process based on the calculated candidate–job matching scores. Displayed matched and missing skills with recommendation results in the Streamlit dashboard and tested the final outputs.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project addressed automated, explainable resume screening and job recommendation by formulating the problem as a five-stage NLP pipeline: text preprocessing, candidate information and skill extraction, job-description analysis, TF-IDF and skill-overlap based matching, and intelligent job recommendation with an administrative panel. The complete system was integrated into an interactive Streamlit application and evaluated on a representative sample resume against a ten-role job catalogue. The captured run shows the pipeline correctly preprocesses and extracts candidate information, ranks all ten job postings by a combined, explainable match score, and recommends the top three roles together with a specific list of matched and missing skills for each. The software implementation is therefore complete and the core objective - transparent, skill-aware resume-job matching - is fully achieved for the evaluated resume, while the report also identifies the skill-dictionary and lexical-matching limitations that should be addressed before broader deployment.

5.2 Future Work

- Replace or augment the fixed skill dictionary with a larger, extensible skill taxonomy, or use semantic embeddings (e.g. Sentence-BERT) to capture synonym and paraphrase similarity beyond exact keyword overlap.
- Extend the system to batch-process and rank multiple resumes for a recruiter-facing shortlisting workflow, rather than a single candidate-facing recommendation view.
- Validate the pipeline on a larger, more diverse set of real resumes and job postings to obtain statistically meaningful matching-accuracy figures.
- Add fairness/bias auditing to check that the skill dictionary and scoring weights do not systematically disadvantage any group of candidates.
- Support additional resume formats (e.g. scanned/image-based PDFs via OCR) and multi-language resumes.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Building a complete NLP preprocessing pipeline (extraction, cleaning, tokenisation, stop-word removal) for unstructured resume text.
- Practical experience combining TF-IDF/cosine-similarity based lexical matching with structured skill-overlap scoring to produce an explainable ranking.
- End-to-end deployment of a multi-module NLP application as an interactive web application using Streamlit.

Engineering & Professional Skills Developed:

- Systematic requirements capture, trade-off analysis and risk assessment for an NLP-based software engineering project.
- Structured testing and verification of a data-driven system against defined acceptance criteria.
- Technical documentation and team-based project planning and reporting.

Individual Reflection - A. Sai Rohit (Module-5):

My major contribution to the project was the development and integration of **Module 5 – Intelligent Job Recommendation and Results**, which forms the final stage of the NLP-driven resume screening system. I worked on converting the resume–job matching results generated by the previous modules into meaningful and personalized job recommendations. I implemented the recommendation process to compare the calculated matching scores of the candidate with the available job descriptions, rank the job roles according to their suitability, and identify the **top three job recommendations** for each candidate. I also worked on organizing the final results so that the recommended job title and corresponding match score could be displayed clearly to the user.

I further contributed to the **matched and missing skill analysis**, where the candidate's extracted skills are compared with the required skills of each recommended job. The system displays the skills that match the job requirements and identifies the skills that are missing, helping the candidate understand their suitability and areas where additional skills may be required. I also contributed to integrating these outputs into the **Streamlit dashboard**, including the presentation of recommendation results, match scores, matched skills, missing skills, and job-related information in an interactive and user-friendly manner. I performed testing using different resumes and job descriptions to verify the ranking of recommendations, correctness of skill matching, and proper display of results. Through this module, I contributed to making the overall system more practical by transforming the NLP and matching outputs into understandable job recommendations that can support candidates in identifying suitable career opportunities.

🏆 Top 3 Job Recommendations

 **Data Scientist**
Match Score: 31.2%

✔ Matched Skills:

✖ Missing Skills:

Python Machine Learning SQL Pandas NumPy Data Visualization

R Statistics TensorFlow Deep Learning

💡 Recommendation: This role may require significant skill development. Key areas to develop include R, Statistics, TensorFlow.

> View additional details

 **Python Developer**
Match Score: 26.2%

✔ Matched Skills:

✖ Missing Skills:

Python SQL Git HTML CSS JavaScript

Django Flask REST APIs PostgreSQL

REFERENCES

1. **An Advanced Real-Time Job Recommendation System and Resume Analyser** – IEEE, 2026
[IEEE Xplore – Job Recommendation & Resume Analysis](#)
2. **AI-driven Semantic Similarity-based Job Matching Framework for Recruitment Systems** – Information Sciences
[ScienceDirect – Semantic Job Matching](#)
3. **Nexus: A Multi-Modal Framework for Semantic Job Matching and AI-Driven Resume Analysis** – Procedia Computer Science, 2026
[ScienceDirect – Nexus Resume Analysis Framework](#)
4. **AI-Enabled Employment Matching Platform** – International Journal on Advanced Computer Theory and Engineering, 2026
[AI-Enabled Employment Matching Platform](#)
5. **Job Recommendation System Using Machine Learning and NLP** – IRJIET, 2026
[Job Recommendation System Using ML and NLP](#)
6. **Automated Resume Screening and Job Recommendation System Using NLP and Deep Learning**
[Automated Resume Screening & Job Recommendation](#)
7. **Skill-Based Resume Evaluation and Job Recommendation System** – IJIRT, 2025
[Skill-Based Resume Evaluation & Job Recommendation](#)
8. **An AI-Based Applicant Tracking System for Resume Analysis and Skill Gap Prediction Using NLP Techniques**
[AI-Based ATS for Resume Analysis & Skill Gap Prediction](#)
9. **ESCO – European Skills, Competences, Qualifications and Occupations** – European Commission
[Official ESCO Classification](#)
Useful for **skill extraction, occupation classification and job matching.**

APPENDICES

Appendix A - Detailed Calculations

A.1 TF-IDF Weighting

For a term t in a document d drawn from the resume/job-description corpus, the TF-IDF weight is: $TFIDF(t, d) = TF(t, d) \times \log(N / DF(t))$, where $TF(t, d)$ is the frequency of term t in document d , N is the total number of documents (resume + all job descriptions) and $DF(t)$ is the number of documents containing t . This up-weights terms that are frequent in a specific document but rare across the whole corpus.

A.2 Cosine Similarity (Lexical / TF-IDF Score)

The lexical similarity between the resume vector R and a job-description vector J_k is: $TFIDF_score(R, J_k) = \cos(\theta) = (R \cdot J_k) / (\|R\| \times \|J_k\|)$, which returns a value between 0 (no lexical overlap) and 1 (identical term-weight vectors).

A.3 Skill-Overlap Score

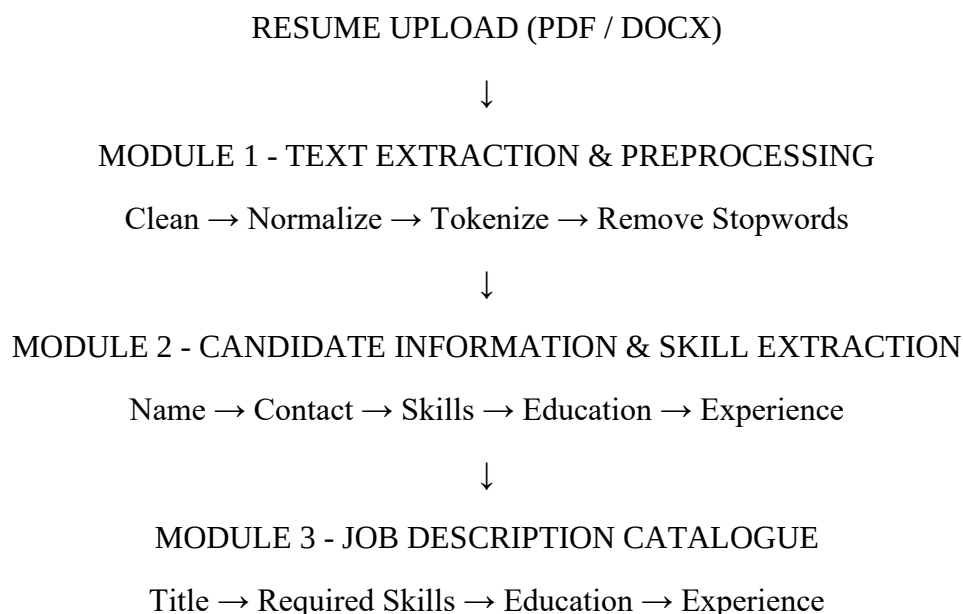
The structured skill-overlap score is: $Skill_score(R, J_k) = |Skills(R) \cap Skills(J_k)| / |Skills(J_k)|$, i.e. the fraction of a job's required skills that are present in the candidate's extracted skill set.

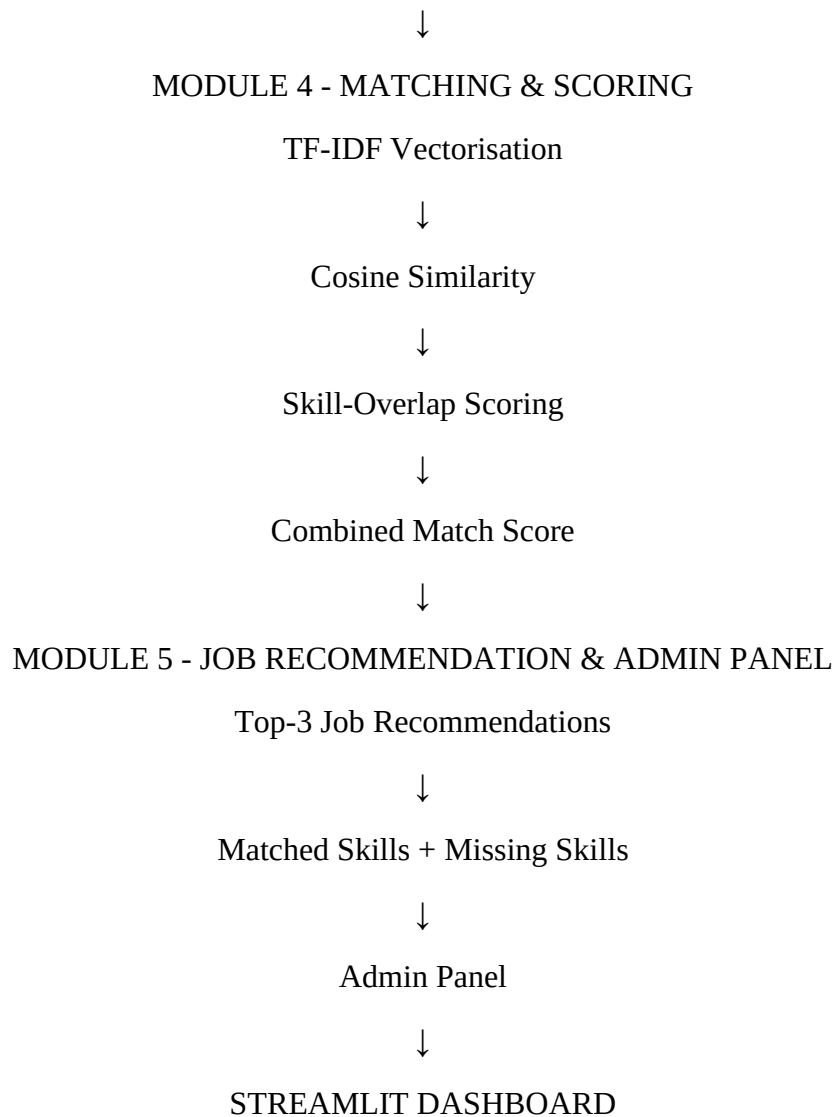
A.4 Combined Match Score

The overall ranking score used in Module 4/5 is: $Match_score(R, J_k) = w_1 \cdot TFIDF_score(R, J_k) + w_2 \cdot Skill_score(R, J_k)$, with $w_1 + w_2 = 1$; the weighting is chosen so that skill overlap, being the more reliable signal for technical roles, contributes more strongly to the final ranking than the lexical TF-IDF term alone.

Appendix B - Engineering Drawings / Schematics

B.1 Detailed System Data Flow





This architecture shows the dependency between the five modules: Module 1 supplies clean text to Module 2; Module 2's candidate profile and Module 3's job catalogue are the two inputs to Module 4's matching engine; Module 4's ranked scores drive the Module 5 recommendation and admin views.

Appendix C - Source Code / Code Repository Information

C.1 Source Code Organisation

- Resume Upload & Text Extraction - reads PDF/DOCX files and returns raw text (Module 1).
- Preprocessing Pipeline - cleaning, normalisation, tokenisation and stop-word removal (Module 1).
- Candidate Information Extractor - pattern- and dictionary-based extraction of contact fields, skills, education and experience (Module 2).
- Job Catalogue Manager - stores, lists and edits job-description records (Module 3).

- Matching Engine - TF-IDF vectorisation, cosine similarity and skill-overlap scoring, producing the combined match score (Module 4).
- Recommendation & Admin Panel - selects the top-3 roles, computes matched/missing skills, and provides add/delete job and candidate-history controls (Module 5).
- Streamlit User Interface - provides the upload control, tabs for each module, and result tables/cards.

The repository link should be added here when the final source repository is made available. Only the implementation structure is documented in this appendix because the complete source code is maintained separately from the report.

Appendix D - Datasheets

Not Applicable. This capstone is a software-based NLP resume-screening and job-recommendation system. No physical sensors, electronic components or hardware equipment were used; therefore, no hardware datasheets are required.

Appendix E - Experimental / Raw Data

E.1 Captured Preprocessing Statistics (Module 1)

Metric	Value
Raw characters	1,986
Characters after cleaning	1,974
Total tokens	275
Tokens after stop-word removal	202

E.2 Captured Resume-Job Match Results (Module 4)

Rank	Job Title	Match Score	TF-IDF Score	Skill Score	Skills Matched
1	Data Scientist	31.2%	12.1%	60.0%	6/10
2	Python Developer	26.2%	3.7%	60.0%	6/10
3	Data Analyst	24.2%	7.0%	50.0%	4/8
4	Full Stack Developer	23.6%	3.0%	54.5%	6/11
5	Machine Learning Engineer	20.6%	7.6%	40.0%	4/10
6	NLP Engineer	19.9%	13.2%	30.0%	3/10
7	AI Engineer	18.6%	11.0%	30.0%	3/10

These values are reproduced from the captured Streamlit application output used in the report. They represent the current sample-resume evaluation run and should not be treated as generally representative results without evaluation on additional resumes.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome	Evidence SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3-4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1-SO7	Contribution Statement + Ch. 4 (4.8)